

Chapter 9 Summary

Groups, Rings, and Their Quotients in Lean 4

Reading notes

Abstract

This chapter moves from the abstract algebraic classes of Chapter 8 to concrete Mathlib objects: group and ring homomorphisms, subgroups and ideals, kernels and images, quotient groups/rings, and the isomorphism theorems that relate them, closing with a short tour of polynomial rings. This note restates the chapter's results with corrected mathematical statements and completed proof terms where the source notes contained typos or ambiguous phrasing; every such correction is flagged in an **Audit Note** box.

Contents

Notation Used in This Note	1
1 Group Homomorphisms, Isomorphisms, and Subgroups	1
1.1 Homomorphisms and Isomorphisms	1
1.2 Subgroups	2
2 Kernels, Images, and the Subgroup Lattice	2
3 Structure Theorems: Lagrange and Sylow	3
4 Quotient Groups and Isomorphism Theorems	3
4.1 Quotient by a Normal Subgroup	3
4.2 The First Isomorphism Theorem	4
4.3 Order of a Quotient Group	4
4.4 A Direct-Product Recognition Theorem	4
5 Ring Homomorphisms and Units	5
6 Ideals and Quotient Rings	6
7 Polynomial Rings	7

Notation Used in This Note

Written here	Meaning	Lean / Mathlib notation
$f : G \rightarrow H$	group homomorphism	<code>f : G ->* H</code>
$G \cong H$	group isomorphism	<code>G <=>* H</code>
$f : R \rightarrow S$	ring homomorphism	<code>f : R ->+* S</code>
$R \cong S$	ring isomorphism	<code>R <=>+* S</code>
$G/H, R/I$	quotient group / ring	<code>G / H, R / I</code>
$\ker f, \operatorname{im} f$	kernel, image of a hom.	<code>f.ker, f.range</code>
$R^\times$	the group of units of R	<code>Units R</code>

1 Group Homomorphisms, Isomorphisms, and Subgroups

1.1 Homomorphisms and Isomorphisms

A *group homomorphism* is a Monoid homomorphism between two groups: a function $f : G \rightarrow H$ with $f(xy) = f(x)f(y)$ (identity-preservation then follows automatically).

Monoid / Group homomorphisms and isomorphisms

```
-- applying a bundled MonoidHom
example {M N : Type*} [Monoid M] [Monoid N] (x y : M) (f : M ->* N) :
  f (x * y) = f x * f y := f.map_mul x y

-- building a group hom from a raw function + a preservation proof
example {G H : Type*} [Group G] [Group H] (f : G -> H)
  (h : forall x y, f (x * y) = f x * f y) : G ->* H :=
  MonoidHom.mk' f h

-- a bijective group hom is a group isomorphism
noncomputable example {G H : Type*} [Group G] [Group H]
  (f : G ->* H) (h : Function.Bijective f) : G <=>* H :=
  MulEquiv.ofBijective f h
```

Also recall that `AddCommGroup` lets us discharge routine additive-group identities automatically, e.g. $z + x + (y - z - x) = y$ via the `abel` tactic, the additive-group analogue of `ring`.

1.2 Subgroups

A subset $S \subseteq G$ is a subgroup exactly when it is closed under the group's structure:

1. it contains the identity (0 for `AddGroup`, 1 for `Group`);
2. it is closed under the group operation;
3. it is closed under inverses ($-a$ resp. a^{-1});
4. its elements coerce back into the ambient group (the carrier is genuinely a subset).

Example: $\mathbb{Z}$ as a subgroup of $\mathbb{Q}$

```
example : AddSubgroup Rat where
  -- carrier = { y in Q | exists x in Z, (x : Q) = y }
  carrier := Set.range ((coe) : Int -> Rat)
  add_mem' := by rintro _ _ (n, rfl) (m, rfl); use n + m; simp
  zero_mem' := by use 0; simp
  neg_mem' := by rintro _ (n, rfl); use -n; simp
```

Example: conjugation preserves the subgroup property

For $x \in G$ and a subgroup $H \leq G$, the *conjugate* $xHx^{-1} = \{xhx^{-1} \mid h \in H\}$ is again a subgroup:

```
def conjugate {G : Type*} [Group G] (x : G) (H : Subgroup G) :
  Subgroup G where
  carrier := {a : G | exists h, h in H and a = x * h * x^{-1}}
  one_mem' := by
    dsimp; use 1
    constructor
```

```

    . exact H.one_mem
    . symm; rw [mul_one, mul_inv_cancel]
  inv_mem' := by
    dsimp; rintro x1 (h, h_in, rfl)
    use h-1, H.inv_mem h_in; group
  mul_mem' := by
    dsimp; rintro a b (ha, ha_in, rfl) (hb, hb_in, rfl)
    use ha * hb, mul_mem ha_in hb_in; group

```

2 Kernels, Images, and the Subgroup Lattice

For a homomorphism $f : G \rightarrow H$, its *kernel* $\ker f = \{g \in G \mid f(g) = 1\}$ and *image* $\text{im } f = \{h \in H \mid \exists g, f(g) = h\}$ are subgroups of G and H respectively:

```

example {G H : Type*} [Group G] [Group H] (f : G →* H) (g : G) :
  g in MonoidHom.ker f <=> f g = 1 := f.mem_ker

example {G H : Type*} [Group G] [Group H] (f : G →* H) (h : H) :
  h in MonoidHom.range f <=> exists g : G, f g = h := f.mem_range

```

More generally, a homomorphism $f : G \rightarrow H$ pushes subgroups of G forward (`map`) and pulls subgroups of H back (`comap`); both operations are monotone and respect composition, so together they form a Galois-connection-like pair between the subgroup lattices of G and H :

Operation	Statement	Lean lemma / term
Map	$G' \leq G, f : G \rightarrow H \Rightarrow f(G') \leq H$	<code>Subgroup.map f G'</code>
Comap	$H' \leq H, f : G \rightarrow H \Rightarrow f^{-1}(H') \leq G$	<code>Subgroup.comap f H'</code>
Map monotone	$S \leq T \Rightarrow \text{map } f S \leq \text{map } f T$	unfolds via <code>mem_map</code>
Comap monotone	$S \leq T \Rightarrow \text{comap } f S \leq \text{comap } f T$	unfolds via <code>mem_comap</code>
Map composition	$(\psi \circ \varphi)(S) = \psi(\varphi(S))$	<code>mem_map + ext</code>
Comap composition	$(\psi \circ \varphi)^{-1}(U) = \varphi^{-1}(\psi^{-1}(U))$	<code>mem_comap + ext</code>

3 Structure Theorems: Lagrange and Sylow

Lagrange's Theorem

```

example {G : Type*} [Group G] (G' : Subgroup G) :
  Nat.card G' ∣ Nat.card G := by
  refine (G'.index, ?_)
  rw [mul_comm G'.index]
  exact G'.index_mul_card.symm

```

$|G'|$ divides $|G|$; the witness is the *index* $[G : G']$, since `index_mul_card` gives $[G : G'] \cdot |G'| = |G|$.

Sylow's Theorem

```
example {G : Type*} [Group G] [Finite G] (p : Nat) {n : Nat} [Fact p.
  Prime]
  (hdvd : p ^ n ∣ Nat.card G) : exists K : Subgroup G, Nat.card K =
    p ^ n :=
  Sylow.exists_subgroup_card_pow_prime p hdvd
```

If p^n divides $|G|$ for a prime p , a subgroup of order exactly p^n exists.

Audit Note

The source writes the Lagrange witness as a single term using Lean's infix rewrite-substitution combinator, informally $h \triangleright e$ (meaning: rewrite e using the equation h , i.e. the term-mode equivalent of `rw [h] at e`). The listing above spells this out as an equivalent short tactic proof (`rw`, then `exact`) to keep the document self-contained.

4 Quotient Groups and Isomorphism Theorems

4.1 Quotient by a Normal Subgroup

$H \trianglelefteq G$ (every conjugate of H stays inside H) is exactly the condition needed for the coset space G/H to inherit a group structure from G :

```
example {G : Type*} [Group G] (H : Subgroup G) [H.Normal] : Group (G / H) :=
  inferInstance
```

Proof sketch. Normality of H means $\forall x \in G, h \in H, xhx^{-1} \in H$. Multiplication on G/H is defined by $(a_1H)(a_2H) := (a_1a_2)H$; well-definedness needs

$$a_1H \cdot a_2H = a_1a_2(a_2^{-1}Ha_2)H = a_1a_2H,$$

which holds precisely because normality gives $a_2^{-1}Ha_2 \subseteq H$.

4.2 The First Isomorphism Theorem

For $\varphi : G \rightarrow M$, the induced map $G/\ker \varphi \rightarrow \text{im } \varphi$ is an isomorphism:

```
example {G M : Type*} [Group G] [Group M] (phi : G ->* M) :
  G / MonoidHom.ker phi <=>* MonoidHom.range phi :=
  QuotientGroup.quotientKerEquivRange phi
```

Audit Note

The source states the target type of this example as $G / \ker \text{phi} \rightarrow^* \text{range phi}$ (a mere homomorphism, $\rightarrow^*$). Since `QuotientGroup.quotientKerEquivRange` constructs a bijective correspondence and the surrounding text calls this fact “Group Equivalence,” the type should be the isomorphism arrow $\lt;=>^*$, as corrected above.

4.3 Order of a Quotient Group

If $|G| = |H| \cdot |K|$ then $|G/H| = |K|$, by cancelling $|H|$ out of $|G/H| \cdot |H| = |G| = |K| \cdot |H|$:

```
lemma aux_card_eq [Finite G] (h' : Nat.card G = Nat.card H * Nat.card K)
  :
  Nat.card (G / H) = Nat.card K := by
```

```

have : Nat.card (G / H) * Nat.card H = Nat.card K * Nat.card H := calc
  Nat.card (G / H) * Nat.card H = Nat.card G           := by rw [<-
    index_eq_card H, index_mul_card H]
  -
    = Nat.card K * Nat.card H := by rw [h
    ', mul_comm]
exact Nat.eq_of_mul_eq_mul_right Nat.card_pos this

```

using `index_eq_card H` ($|G : H| = |G/H|$) and `index_mul_card H` ($|G : H| \cdot |H| = |G|$).

4.4 A Direct-Product Recognition Theorem

The source’s final three results are labeled “First/Second/Third Group Isomorphism Theorem,” but they do not correspond to the classically-numbered isomorphism theorems (whose usual statements are $G/\ker \varphi \cong \text{im } \varphi$; the diamond theorem $HN/N \cong H/(H \cap N)$; and $(G/N)/(M/N) \cong G/M$). They are instead the three steps of a single argument — under the hypotheses $H, K \leq G$, $|G| = |H| \cdot |K|$, and $H \cap K = \{1\}$ — that G is (isomorphic to) the internal direct product $H \times K$.

Audit Note

The source also describes the hypothesis as “ H, G are disjoint,” which cannot be literal (a subgroup always contains G ’s identity, so H and G trivially intersect at 1, and $H \subseteq G$ regardless). The intended hypothesis, consistent with the Lean proof terms, is that H and K intersect trivially, $H \cap K = \{1\}$; the phrasing below uses that reading.

Step 1: $K \cong G/H$

```

def iso1 : K <=>* G / H := by
  apply MulEquiv.ofBijective ((QuotientGroup.mk' H).restrict K)
  rw [Nat.bijective_iff_injective_and_card]
  constructor
  . rw [<- ker_eq_bot_iff, (QuotientGroup.mk' H).ker_restrict K];
    simp [h]
  . symm; exact aux_card_eq h'

```

`QuotientGroup.mk' H` is the canonical hom $G \rightarrow G/H$; restricting it to K is injective exactly when $K \cap H = \{1\}$, and `Nat.bijective_iff_injective_and_card` upgrades injective + matching cardinalities to bijective.

Step 2: $G \cong (G/K) \times (G/H)$

```

def iso2 : G <=>* (G / K) * (G / H) := by
  -- the pair of canonical projections (G -> G/K, G -> G/H) combined
  apply MulEquiv.ofBijective <| (QuotientGroup.mk' K).prod (
    QuotientGroup.mk' H)
  rw [Nat.bijective_iff_injective_and_card]
  constructor
  . rw [<- ker_eq_bot_iff, ker_prod]; simp [h.symm.eq_bot]
  . rw [Nat.card_prod, aux_card_eq h', aux_card_eq (mul_comm (Nat.
    card H) _ substEq h'), h']

```

The product of the two canonical projections is injective because the kernels’ intersection is trivial, and the cardinalities match by hypothesis. (`substEq` stands in for Lean’s built-in term-level rewrite-substitution combinator, spelled out as an identifier here to keep the listing plain ASCII.)

Step 3: the recognition theorem $G \cong H \times K$

```
def finalIso : G <=>* H * K :=
  (iso2 h h').trans (((iso1 h.symm (mul_comm (Nat.card H) _ substEq h
    ')).prodCongr
    (iso1 h h')).symm)
```

Chain: Step 2 reduces $G \cong H \times K$ to $G \cong (G/K) \times (G/H)$; Step 1 (applied twice, with roles of H, K swapped) turns $(G/K) \times (G/H)$ into $H \times K$; composing gives $G \cong H \times K$.

5 Ring Homomorphisms and Units

Facts 1--4: units of a ring

```
-- Fact 1: the only units of Z are 1 and -1
example (x : Units Int) : x = 1 or x = -1 := Int.units_eq_one_or x

-- Fact 2: a unit times its inverse is 1 (Monoid is multiplicative by
  default)
example {M : Type*} [Monoid M] (x : Units M) : (x : M) * x^{-1} = 1
:= Units.mul_inv x

-- Fact 3: the units of a monoid form a group
example {M : Type*} [Monoid M] : Group (Units M) := inferInstance

-- Fact 4: a ring hom induces a hom between unit groups
example {R S : Type*} [Ring R] [Ring S] (f : R ->+* S) : Units R ->+*
  Units S :=
  Units.map f
```

6 Ideals and Quotient Rings

Facts 5--8: the quotient ring R/I

```
-- Fact 5: every ideal gives a canonical quotient ring hom
example {R : Type*} [CommRing R] (I : Ideal R) : R ->+* R / I :=
  Ideal.Quotient.mk I

-- Fact 6: elements of I are exactly the elements sent to 0
example {R : Type*} [CommRing R] {a : R} {I : Ideal R} :
  Ideal.Quotient.mk I a = 0 <-> a in I :=
  Ideal.Quotient.eq_zero_iff_mem

-- Fact 7: universal property -- f factors through R / I whenever I
  <= ker f
example {R S : Type*} [CommRing R] [CommRing S] (I : Ideal R) (f : R
  ->+* S)
  (H : I <= RingHom.ker f) : R / I ->+* S :=
  Ideal.Quotient.lift I f H

-- Fact 8: the (ring) First Isomorphism Theorem
example {R S : Type*} [CommRing R] [CommRing S] (f : R ->+* S) :
  R / RingHom.ker f <=>+* f.range :=
  RingHom.quotientKerEquivRange f
```

Fact 5 is the ring analogue of `QuotientGroup.mk'`; Fact 7 is the universal property that makes R/I the “most efficient” quotient satisfying $I \subseteq \ker f$; Fact 8 is the ring version of the group isomorphism theorem from Section 5.2.

Facts 9--14: ideal arithmetic and quotient maps

For ideals I, J of a commutative ring R :

Fact	Statement	Lean lemma / term
9	$x \in I + J \iff \exists a \in I, b \in J, a + b = x$	<code>Submodule.mem_sup</code>
10	$I \cdot J \subseteq J$	<code>Ideal.mul_le_left</code>
11	$I \cdot J \subseteq I$	<code>Ideal.mul_le_right</code>
12	$I \cdot J \subseteq I \cap J$	<code>Ideal.mul_le_inf</code>
13	$I \subseteq f^{-1}(J) \Rightarrow$ exists a lift $R/I \rightarrow S/J$	<code>Ideal.quotientMap</code>
14	$I = J \Rightarrow R/I \cong R/J$	<code>Ideal.quotEquivOfEq</code>

Key Idea

Facts 10–12 all follow the same pattern: since $\forall x \in R, x \cdot J \subseteq J$ (resp. $I \cdot x \subseteq I$) by definition of ideal multiplication, restricting x to range over the smaller set I (resp. J) only shrinks the union, giving both containments and hence containment in the intersection $I \cap J$.

7 Polynomial Rings

Fact 15: arithmetic, the embedding C , and coefficients in $R[X]$

```
-- C r is the constant polynomial with value r; C : R ->+* R[X] is a
  ring hom
example {R : Type*} [CommRing R] (r : R) : (X + C r) * (X - C r) = X
  ^ 2 - C (r ^ 2) := by
  rw [C.map_pow]; ring

example {R : Type*} [CommRing R] (r : R) : (C r).coeff 0 = r := by
  simp

example {R : Type*} [CommRing R] :
  (X ^ 2 + 2 * X + C 3 : Poly R).coeff 1 = 2 := by simp
```

Because C is a ring homomorphism, $(Cr)^2 = C(r^2)$, which is what finishes the first identity once `ring` handles the remaining polynomial algebra. The coefficient lemmas show `coeff` reading off r from the constant polynomial Cr , and the coefficient of X^1 from $X^2 + 2X + C 3$.

Audit Note

Two issues in the source were corrected above: (1) the header claimed these examples show that “ $R[X]$ is also a commutative ring,” but `CommRing (R[X])` is an existing Mathlib instance, not something these snippets establish — they instead illustrate arithmetic, the embedding C , and `coeff`, so the description here was reworded to match; (2) the proof of the first identity ended with the incomplete tactic `rin`, completed above as `ring` (the tactic needed to close the remaining polynomial-ring equality).

Fact	Statement	Lean lemma / term
16	$\deg(pq) = \deg p + \deg q$ over a semiring with no zero divisors	<code>Polynomial.degree_mul</code>
17	$\deg(p \circ q) = \deg p \cdot \deg q$ (natural-number degree)	<code>Polynomial.natDegree_comp</code>
18	evaluating p at $x \in R$ gives an element of R	<code>P.eval x</code>
19	r is a root of $p \iff p(r) = 0$	<code>Iff.rfl</code>

Audit Note

Fact 19's source proof term was written `Iff.rf`, an incomplete/typo'd form of `Iff.rfl` (the proof that `IsRoot P r` unfolds definitionally to `P.eval r = 0`); corrected above.

Facts 20--21: roots of $(X - Cr)^n$ over an integral domain

```
-- Fact 20: (X - C r) has the single root r
example {R : Type*} [CommRing R] [IsDomain R] (r : R) :
  (X - C r).roots = {r} := roots_X_sub_C r

-- Fact 21: (X - C r)^n has root r with multiplicity n
example {R : Type*} [CommRing R] [IsDomain R] (r : R) (n : Nat) :
  ((X - C r) ^ n).roots = nsmul n {r} : Multiset R := by simp
```

`IsDomain R` (commutative ring, no zero divisors) is required because `.roots` relies on the fact that a nonzero degree- n polynomial has at most n roots *in an integral domain* — this bound fails over rings with zero divisors. In Fact 21, `nsmul n {r}` is the multiset containing r with multiplicity n (Lean writes this with an infix `SMul` operator; `nsmul` is used here purely to keep the listing ASCII-only) — unrelated to the ring's own scalar action.